



PROJECT PLAN

eCMRs plan

SyteLine - one Change Management Request form on its own table and IDO. Live on TRN; next: production

Seven milestones to production

The plan at a glance — each phase in order.

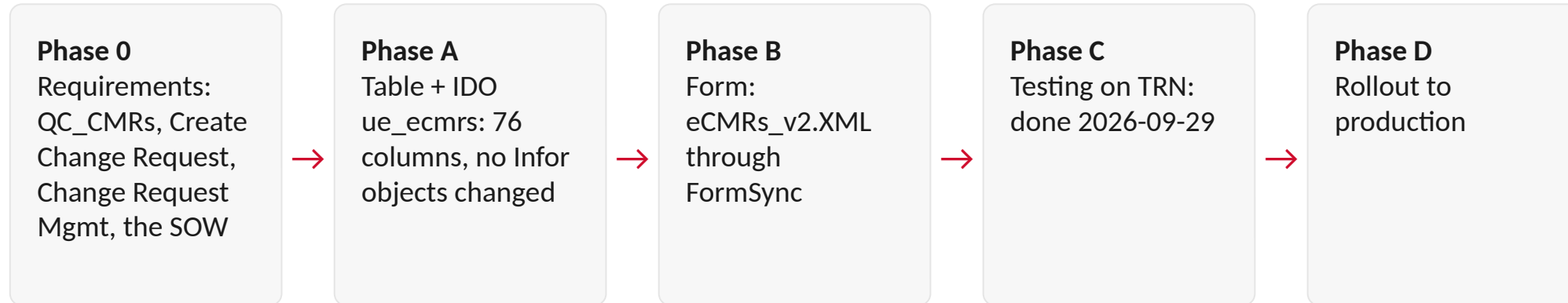
- 1 Scope**
Everything the three legacy CMR screens and the SOW need, on one form.
- 2 Design**
New table and IDO ue_ecmrs, form layout, lookups and scripts.
- 3 Develop**
Built on TRN first — never directly in production. Done.
- 4 Staging**
Tested end to end on TRN; exports checked against the repo. Done 2026-09-29.
- 5 Launch**
Same table, IDO and form in production, from the TRN exports. Next.
- 6 Test**
Full checklist passed on TRN; smoke test in production.
- 7 Optimize**
Two weeks of follow-up with Quality, Engineering, Planning, Purchasing, CM.



Scope

Replace three legacy CMR screens with one form, built on its own table.

The actual phase flow





BRD

Business requirements for the change.

Business requirements

- **One form** for everything the three legacy CMR screens hold, created with **New**
- **Own table and IDO** (ue_ecmrs): no dependency on rs_cmr / rs_crcvr
- **Unique CMR number** CMR-YYMMDD-HHMMSS, set on New; never repeats
- Lookups fill descriptions: Item, Next Assy, Vendor, Dept, Work Center, every **Reviewer ID**
- **Initial Change** ticks the right Req checkboxes; **Reason / Cause Codes** from the QC_MRRs lists
- Ticking **Closed** sets Close Date and Closed By
- No changes to Infor standard objects; Enflite naming (ue_, ENF)
- Built and tested on **TRN** first; delivered through **FormSync**; kept in **GitHub** (Enflite/eCMRs)

TRN first, then production

Everything is built and proven on TRN, then pushed to production later.



On TRN

- 1 Build table and IDO ue_ecmrs
- 2 Set Inline Lists, Read Only, CmrNum; Check In
- 3 Import eCMRs_v2.XML through FormSync; test every field
- 4 Export SQL Columns + IDO Properties as the production source



In production

- 1 Import the same columns and properties from the TRN exports
- 2 Check In; verify production against TRN with one script
- 3 Import the same eCMRs_v2.XML through FormSync
- 4 Smoke test, then hand over to the team



Design

A new table and IDO — nothing Infor owns is changed.

Table and IDO

Object	Name	Notes
SQL table	ue_ecmrs	76 columns (+7 system). Text columns char(255)
Key	cmr_num	nvarchar, Primary Key, no default
IDO	ue_ecmrs (alias ec)	One property per column; lengths equal SQL
Key property	CmrNum	String, 255, Default Value blank
Lists	Inline Lists	Status, Priority, Initial Change, Dispositions, Reason, Cause
Auto-filled	Read Only cleared	Descriptions, reviewer names, Close Date, Closed By

Every column and property: docs/production-build-sheet.md

Design

How the form fills things in — all in eCMRs_v2.XML.



Form mechanisms

On the form	Mechanism
CMR Num on New	StdObjectNewCompleted: CMR- + yyMMdd-HH:mm:ss
Item / Next Assy / Vendor → text	DefaultFrom Item(...), VendNum(VendorName)
Dept / Work Center → description	Validators SetPropertyFromList(..., Description)
Reviewer ID → name (×6)	DefaultFrom EmpNum(<name>) on SLEmployees
Initial Change → Req boxes	DefaultFrom Change(ReqCosting, ...)
Closed → date, user	SetCloseInfo + SetClosedBy scripts

Why each one is built this way, and what failed first: docs/troubleshooting.md



Develop

Build on TRN first — never directly in production.

Actual build steps

- Create table and IDO **ue_ecmrs** in Application Studio (project ue_ENF)
- Import columns and properties in the grids' own CSV format
- Set **Inline Lists**, clear **Read Only** on auto-filled fields, **CmrNum** String / no default
- **Check In** the IDO — edits don't take effect until it is checked in
- Build the form with tools/apply_form_changes.py; import through **FormSync**

Develop

The form is built as XML and imported through FormSync.



Building the form with FormSync

- 1 Keep the TRN v1 form in GitHub as the original (rollback copy)
- 2 `tools/apply_form_changes.py` edits v1 in place into `exports/eCMRs_v2.XML` (`--check`)
- 3 Import `eCMRs_v2.XML` into TRN through FormSync, at Site scope
- 4 Use FormSync to bring the same XML to production at Launch

Staging

Freeze the TRN build and run final validation before production.



Actual staging approach

- 1 Test every field on TRN — passed 2026-09-29
- 2 Export SQL Columns + IDO Properties; `compare_live_lengths.py`: nothing to change
- 3 `make_production_imports.py` builds the production files from those exports
- 4 Team sign-off on TRN



Launch

Deploy to production.

Production steps

- 1 Scheduled window; check ue_ecmrs / eCMRs don't exist in production yet
- 2 New table ue_ecmrs; import exports/production/ue_ecmrs_SqlColumns_import.csv
- 3 New IDO ue_ecmrs (alias ec); import ue_ecmrs_IdoProperties_import.csv; Check In
- 4 Export both grids; make_production_imports.py --verify: production matches TRN
- 5 Import eCMRs_v2.XML through FormSync at Site scope; give users access as on TRN
- 6 Smoke test: New, lookups, cascade, codes, save twice, Closed

Rollback

- New form on a new table — take **eCMRs** off the menu and keep using the legacy CMR screens, which are untouched. No data is lost.

Test

Run tests to make sure everything looks good.



Actual test list

- New gives CMR-YYMMDD-HHMMSS; saves work; no duplicate-key error
- Every lookup fills its description / name; PO Line shows the line number
- Initial Change ticks the right Req boxes; Reason / Cause codes list
- Serial # and LOT # list the item's serials / lots; Closed sets date and user
- Existing records open without errors; team signs off



Optimize

Two weeks of optimization, working closely with the team.

Actual optimization backlog

- Track issues from each team using eCMRs; fix on TRN first
- Decide: Notify, Reported By lookup, Assigned Buyer, Sub Assembly
- Retire the legacy CMR screens; decide on history migration
- Update the CMR procedure (QA-300-037); manager approval